```
"""
test_pipeline.py
================
Generates synthetic BirdTrack-style data and validates the deduplication
pipeline against known expected outcomes.

Run with: python test_pipeline.py
"""

import pandas as pd
import numpy as np
import os
import sys
import tempfile
from datetime import datetime, timedelta

# Add parent dir to path
sys.path.insert(0, os.path.dirname(__file__))
from dedup import (
    load_birdtrack, dedupe_same_visit, cluster_species,
    estimate_counts, annual_summary, build_review_table
)
from config import COLUMN_MAP, BEHAVIOUR_WINDOWS
from species_config import load_species_config

PASS = "✓"
FAIL = "✗"
results = []


def check(name, condition, detail=''):
    status = PASS if condition else FAIL
    results.append((status, name, detail))
    print(f"  {status}  {name}" + (f"  [{detail}]" if detail else ''))


# ---------------------------------------------------------------------------
# Synthetic data builder
# ---------------------------------------------------------------------------

def make_record(species, date, site, lat, lon, observer, count,
                visit_id=None, time=None):
    return {
        'Species':          species,
```

```
        'Date':             date,
        'Site name':        site,
        'Latitude':         lat,
        'Longitude':        lon,
        'Recorder name':    observer,
        'Count':            count,
        'Visit ID':         visit_id or '',
        'Start time':       time or '09:00',
    }


def write_csv(records, path):
    pd.DataFrame(records).to_csv(path, index=False)


def write_species_config(species_list, path):
    rows = [{'species': s, 'behaviour': b, 'radius_km': '', 'time_window_hours':
            for s, b in species_list]
    pd.DataFrame(rows).to_csv(path, index=False)


# ---------------------------------------------------------------------------
# Test cases
# ---------------------------------------------------------------------------

def test_same_visit_dedup():
    """Two observers on same visit to same site: keep max count."""
    print("\nTest 1: Same-visit deduplication")
    records = [
        make_record('Marsh Warbler', '2023-06-01', 'Lakeside', 52.0, -1.0,
                    'ObsA', 1, visit_id='V001'),
        make_record('Marsh Warbler', '2023-06-01', 'Lakeside', 52.0, -1.0,
                    'ObsB', 2, visit_id='V001'),  # same visit
    ]
    with tempfile.TemporaryDirectory() as tmpdir:
        csv = os.path.join(tmpdir, 'records.csv')
        write_csv(records, csv)
        df = load_birdtrack(csv, COLUMN_MAP)
        deduped = dedupe_same_visit(df)
        check("Two records with same visit_id reduced to one",
              len(deduped[deduped['species'] == 'Marsh Warbler']) == 1)
        check("Kept max count (2)",
              deduped[deduped['species'] == 'Marsh Warbler']['count_int'].iloc[0]


def test_same_site_different_observers():
    """Two observers, no visit ID, same site same day: should be flagged."""
```

```
    print("\nTest 2: Same site, different observers — flagged for review")
    records = [
        make_record('Marsh Sandpiper', '2023-08-15', 'East Marsh', 52.1, -0.9,
                    'ObsA', 1, time='09:00'),
        make_record('Marsh Sandpiper', '2023-08-15', 'East Marsh', 52.1002, -0.90
                    'ObsB', 1, time='10:30'),
    ]
    with tempfile.TemporaryDirectory() as tmpdir:
        csv = os.path.join(tmpdir, 'records.csv')
        write_csv(records, csv)
        df = load_birdtrack(csv, COLUMN_MAP)
        windows = BEHAVIOUR_WINDOWS['mobile']
        clustered = cluster_species(df[df['species'] == 'Marsh Sandpiper'],
                                    windows['radius_km'],
                                    windows['time_window_hours'])
        check("Two nearby records in same cluster",
              clustered['cluster_id'].nunique() == 1)
        review = build_review_table(clustered, 'Marsh Sandpiper', 'mobile')
        check("Multi-observer cluster flagged for review",
              len(review) == 2)
        summary = estimate_counts(clustered)
        check("Min individuals = 1 (max of counts)",
              summary['min_individuals'].iloc[0] == 1)


def test_genuinely_different_individuals():
    """Two records at distant sites same day: should be separate clusters."""
    print("\nTest 3: Distant sites — separate individuals")
    records = [
        make_record('Waxwing', '2023-11-20', 'North Park', 52.0, -1.0,
                    'ObsA', 5, time='09:00'),
        make_record('Waxwing', '2023-11-20', 'South Retail', 52.1, -1.5,
                    'ObsB', 12, time='11:00'),  # ~30km away
    ]
    with tempfile.TemporaryDirectory() as tmpdir:
        csv = os.path.join(tmpdir, 'records.csv')
        write_csv(records, csv)
        df = load_birdtrack(csv, COLUMN_MAP)
        windows = BEHAVIOUR_WINDOWS['irruptive']
        clustered = cluster_species(df[df['species'] == 'Waxwing'],
                                    windows['radius_km'],
                                    windows['time_window_hours'])
        check("Distant records in separate clusters",
              clustered['cluster_id'].nunique() == 2)
        annual = annual_summary(clustered, 'Waxwing', 'irruptive')
        check("Min individuals = 17 (5 + 12)",
              annual['min_individuals'].iloc[0] == 17)
```

```
def test_returning_sedentary_bird():
    """Same bird reported across multiple days at same location."""
    print("\nTest 4: Returning sedentary bird — one individual across days")
    records = [
        make_record('Great Snipe', '2023-09-01', 'Thornham Marsh', 52.96, 0.57,
                    'ObsA', 1, time='07:00'),
        make_record('Great Snipe', '2023-09-02', 'Thornham Marsh', 52.9601, 0.570
                    'ObsB', 1, time='08:00'),
        make_record('Great Snipe', '2023-09-03', 'Thornham Marsh', 52.9600, 0.570
                    'ObsC', 1, time='07:30'),
    ]
    with tempfile.TemporaryDirectory() as tmpdir:
        csv = os.path.join(tmpdir, 'records.csv')
        write_csv(records, csv)
        df = load_birdtrack(csv, COLUMN_MAP)
        windows = BEHAVIOUR_WINDOWS['sedentary']
        clustered = cluster_species(df[df['species'] == 'Great Snipe'],
                                    windows['radius_km'],
                                    windows['time_window_hours'])
        # 3 separate days = 3 temporal clusters (bird reported on different days)
        # Each is a separate detection event, but min individuals = 1 per cluster
        annual = annual_summary(clustered, 'Great Snipe', 'sedentary')
        check("Min individuals = 1 (same bird each day)",
              annual['min_individuals'].iloc[0] == 1)
        check("Max individuals = 3 (could be different birds each day)",
              annual['max_individuals'].iloc[0] == 3)


def test_count_x_handling():
    """Records with 'X' count should be treated as 1."""
    print("\nTest 5: 'X' count handling")
    records = [
        make_record('Pallid Swift', '2023-05-10', 'Coastal Point', 51.0, 1.2,
                    'ObsA', 'X'),
        make_record('Pallid Swift', '2023-05-10', 'Coastal Point', 51.0002, 1.200
                    'ObsB', 'X'),
    ]
    with tempfile.TemporaryDirectory() as tmpdir:
        csv = os.path.join(tmpdir, 'records.csv')
        write_csv(records, csv)
        df = load_birdtrack(csv, COLUMN_MAP)
        check("'X' counts converted to 1",
              (df['count_int'] == 1).all())
```

```
def test_no_coordinates():
    """Records without lat/lon should cluster by site name."""
    print("\nTest 6: Records without coordinates — cluster by site name")
    records = [
        make_record('Aquatic Warbler', '2023-07-15', 'Rye Harbour', np.nan, np.na
                    'ObsA', 1),
        make_record('Aquatic Warbler', '2023-07-15', 'Rye Harbour', np.nan, np.na
                    'ObsB', 1),
        make_record('Aquatic Warbler', '2023-07-15', 'Dungeness', np.nan, np.nan,
                    'ObsC', 1),
    ]
    with tempfile.TemporaryDirectory() as tmpdir:
        csv = os.path.join(tmpdir, 'records.csv')
        write_csv(records, csv)
        df = load_birdtrack(csv, COLUMN_MAP)
        windows = BEHAVIOUR_WINDOWS['sedentary']
        clustered = cluster_species(df[df['species'] == 'Aquatic Warbler'],
                                    windows['radius_km'],
                                    windows['time_window_hours'])
        # Rye Harbour and Dungeness = 2 spatial clusters
        n_spatial = clustered['spatial_cluster'].nunique()
        check("Two site names = two clusters",
              n_spatial == 2, f"got {n_spatial}")


# ---------------------------------------------------------------------------
# Run all tests
# ---------------------------------------------------------------------------

if __name__ == '__main__':
    print("=" * 55)
    print("BirdTrack Deduplication Pipeline — Test Suite")
    print("=" * 55)

    test_same_visit_dedup()
    test_same_site_different_observers()
    test_genuinely_different_individuals()
    test_returning_sedentary_bird()
    test_count_x_handling()
    test_no_coordinates()

    print("\n" + "=" * 55)
    passed = sum(1 for r in results if r[0] == PASS)
    failed = sum(1 for r in results if r[0] == FAIL)
    print(f"Results: {passed} passed, {failed} failed")
    if failed:
        print("\nFailed tests:")
```

```
    for r in results:
        if r[0] == FAIL:
            print(f"  {r[1]}")
    sys.exit(1)
else:
    print("All tests passed.")
```